Raspberry Pi Pico でセンサーを動かそう

～信号機・人感センサー・超音波センサー～



今日のゴール

1. Pico で **LED を光らせる** / **センサーの値を読む**
2. `while` / `if` / `for` / 関数 を使って「動きの流れ」を作れる
3. 電気の基本 (**抵抗・電圧・プルアップ**) がなぜ必要か説明できる

Raspberry Pi Pico と GPIO

- **マイコン** ＝ 小さなコンピュータ
- **GPIO** ＝「Pico が電気を ON/OFF したり、読み取ったりできる出入口」
  - 出力 (OUT): 電気を出す 例)LED を光らせる
  - 入力 (IN): 電気を読み取る 例) ボタンを調べる
- **3.3V が HIGH(1)、0V が LOW(0)**

番号の種類	意味	例
GP 番号	プログラムで使う番号	`Pin(3, Pin.OUT)` → GP3
物理ピン番号	基板上の位置 (1 ～ 40)	GP3 は物理 5 番

⚠ 物理 3 番は GND であって GP3 ではない！ 配布したピン配置図で、GP 番号と物理ピン番号の両方を確かめよう

Thonny の準備

- Pico を USB でパソコンにつなぐ
- 右下のインタプリタを **「MicroPython (Raspberry Pi Pico)」** にする
- Shell に `>>>` が出ればつながっている

ボタン	意味
▶ 実行 (緑の三角)	プログラムを Pico で動かす
■ 停止 (赤い四角)	動いているプログラムを止める
Shell(画面下)	`print` の結果が表示される

⚠ Pico が見つからない → USB ケーブルが「充電専用」でないか確認

まずは LED をつけてみよう ①つける

```
from machine import Pin          # Pin:Pico のピンを操作するための道具

led = Pin("LED", Pin.OUT)        #  本体の LED を「出力」にする
led.value(1)                     # 1 = 点灯
```

- `Pin("LED", Pin.OUT)`: 本体の LED を **出力** にする
- `led.value(1)`:**1 で点灯**
- やってみよう :`1` を `0` にすると ?(→ 消える)

まずは LED をつけてみよう ②つけて消す

```
from machine import Pin          # Pin:Pico のピンを操作するための道具
import time                      # time: 待ち時間を作るための道具

led = Pin("LED", Pin.OUT)        # 本体の LED を「出力」にする
led.value(1)                     # 点灯
time.sleep(1)                    # 1 秒待つ
led.value(0)                     # 0 = 消灯
```

- `time.sleep(1)`:**1 秒待つ**
- 上から順に : 点灯 → 待つ → 消灯
- やってみよう :`1` を `3` にすると ?(→ 3 秒光る)

まずは LED をつけてみよう ③点滅

```
from machine import Pin          # Pin:Pico のピンを操作するための道具
import time                      # time: 待ち時間を作るための道具

led = Pin("LED", Pin.OUT)        # 本体の LED を「出力」にする

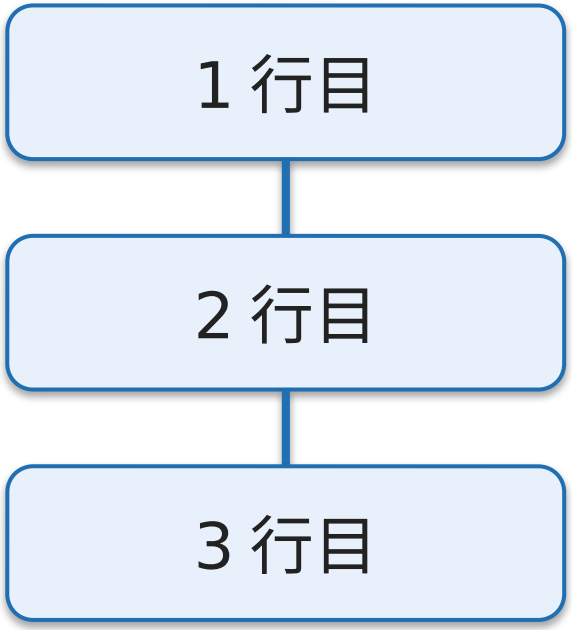
while True:                      # ずっとくり返す ( 止めるときは停止ボタン )
    led.value(1)                 # 点灯
    time.sleep(0.5)              # 0.5 秒待つ
    led.value(0)                 # 消灯
    time.sleep(0.5)              # 0.5 秒待つ
```

- `while True:` の中 (字下げした 4 行) が **ずっとくり返される**
- 止めるときは **停止ボタン (■)**
- やってみよう :`0.5` を `0.1` にすると ?(→ 速く点滅)

色付き枠 : **while**

プログラムは上から順に動く（順次）

```
print("1行目")
print("2行目")    # 1行目のあとに実行される
print("3行目")    # 2行目のあとに実行される
```



- **1行ずつ、上から順番に**実行される
- 順番を入れかえると、動きも変わる

while True: 無限ループ

```
while True:                 # True なので終わらない
    print("ずっと表示")
    time.sleep(1)           # 1秒待つ
```

- マイコンは **「ずっと動き続ける」** のが基本
- `while True:` の中は、終わらずにくり返される
- 止めるには Thonny の **停止ボタン (■)**

色付き枠 : **while**

if 条件分岐

```
if distance < 10:          #  もし距離が 10cm 未満な
ら
    print("近い!")
else:                      #  そうでなければ
    print("遠い")
```

- 「**もし** ボタンが押されていたら」
- 「**もし** 距離が 10cm 未満なら」
- 条件によって、する処理を分ける

色付き枠： if

for 繰り返し

```
for i in range(3):                # i に 0, 1, 2 が順に入る
    print(i)

for p in (red, yellow, green):    # LED を 1 個ずつ取り出す
    p.value(1)
```

- **決まった回数** をくり返す
- **決まったもの** を順に処理する (LED を 1 個ずつ点灯)

色付き枠 : for

関数 def

```
def double(x):
    return x * 2          # x を 2 倍した結果を返す

print(double(5))          # 10 と表示される
```

- よく使う処理に **名前を付ける**
  - 例 :`all_off()`(全部消す)、 `read_cm()`(距離を測る)
- `return` で **結果を返す**

色付き枠 : **def**

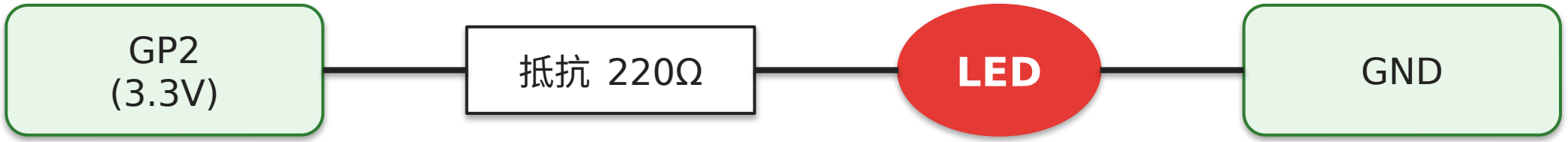
time.sleep() 待つ命令

書き方	待つ時間
`time.sleep(1)`	1 秒
`time.sleep_ms(100)`	100 ミリ秒 ＝ 0.1 秒
`time.sleep_us(10)`	10 マイクロ秒 ＝ 0.00001 秒

★ 待っている間は **他のことができない！**
→ あとのボタン・割り込みで、この問題が出てくるよ

LED は抵抗なしでつないではいけない

- LED は **電流が流れすぎると壊れる** (最悪の場合 Pico 側も傷む)
- 電流を抑えるために **電流制限抵抗** を **直列** (一列) に入れる
- 単体 LED を光らせるときは **220Ω 程度の抵抗が必要**



抵抗値の計算 (オームの法則)

オームの法則 :V = I × R → I = V ÷ R LED は光るとき約 **2V** を使う (順方向電圧 Vf ・赤 LED の目安)

① 抵抗にかかる電圧

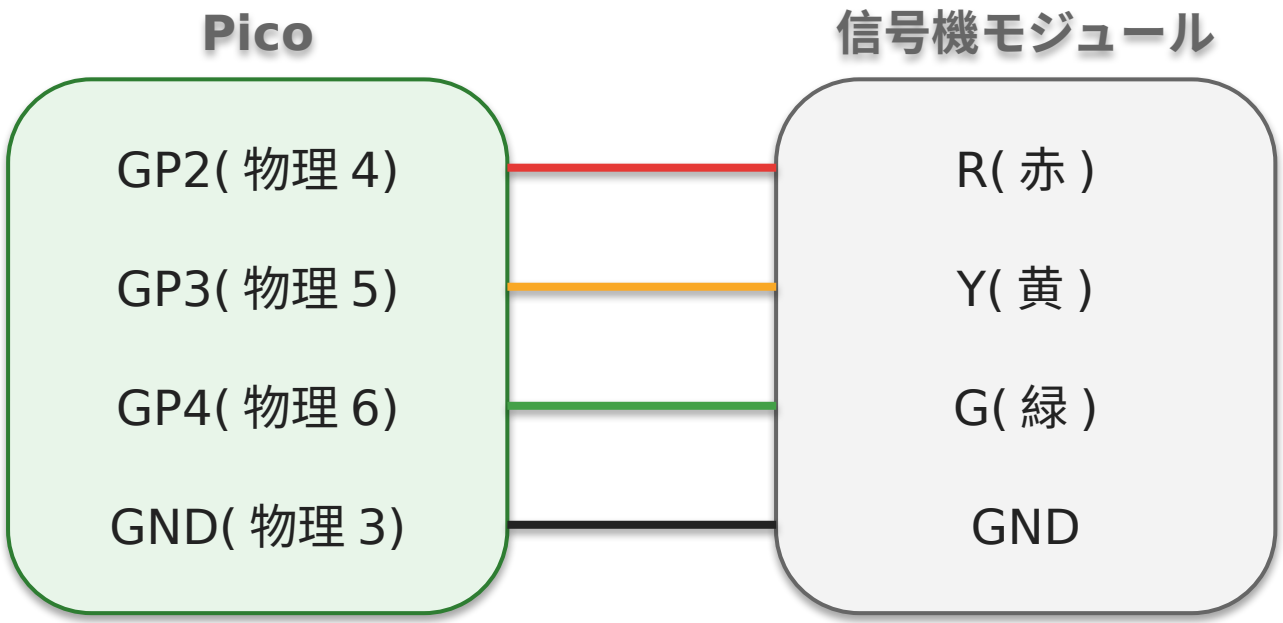
式	抵抗の電圧 = Pico の電圧 − LED の電圧
数値を入れる	3.3V − 2.0V
答え	**1.3V**

② 流れる電流

式	I = V ÷ R
数値を入れる	1.3V ÷ 220Ω
答え	**≒ 0.006A = 6mA**

⚠ 6mA は安全な範囲 (LED は目安 20mA 以下、 Pico の GPIO は控えめに)。
抵抗なしだと？→ 電流を止めるものがなく、大きな電流が流れてしまう！

今日の信号機モジュールは抵抗入り



- 基板に **抵抗が組み込まれている** → GPIO に **直結 OK**
- 部品を単体で使うときは **220Ω を入れる**
- GND は **物理 3 番** (GP3 ではない !)

★ 配布したピン配置図で、 GP 番号と物理ピン番号の両方を確かめよう

信号機コード (配線チェック + サイクル)(1/3)

```
# ----  必要な道具 ( モジュール ) を読み込む ----
from machine import Pin    # Pin:Pico  のピンを操作するための道具
import time                # time: 待ち時間を作るための道具

# ----  ピンの設定 ----
# Pin( 番号 , Pin.OUT)  で「このピンは電気を出す ( 出力 )」と決める
red    = Pin(2, Pin.OUT)   # GP2  に赤の LED
yellow = Pin(3, Pin.OUT)   # GP3  に黄の LED
green  = Pin(4, Pin.OUT)   # GP4  に緑の LED

# ----  関数 : 全部の LED を消す ----
def all_off():
    #  赤・黄・緑を 1 つずつ取り出して p  に入れ、順番に消す
    for p in (red, yellow, green):
        p.value(0)         # 0 =  電気を止める =  消灯
```

→ 次のスライドに続く

色付き枠 : for def

信号機コード (配線チェック + サイクル)(2/3)

```
# ----  配線チェック :1 個ずつ点灯 ----
# ( 名前 ,  ピン )  の組を順番に取り出して処理する
for name, p in (("赤", red), ("黄", yellow), ("緑", green)):
    all_off()                 #  いったん全部消す
    print(name, "点灯")       # Shell  に今どの色かを表示
    p.value(1)                # 1 =  電気を流す =  点灯
    time.sleep(1)             # 1 秒待つ
all_off()                     #  チェックが終わったら全部消す
```

→ 次のスライドに続く

色付き枠 : for

信号機コード (配線チェック + サイクル)(3/3)

```
# ---- 信号のサイクル ( ずっと繰り返す ) ----
while True                    # True なので終わらない ( 無限ループ )
    green.value(1)            #  緑を点ける
    time.sleep(5)             # 5 秒待つ
    green.value(0)            #  緑を消す

    yellow.value(1)           #  黄を点ける
    time.sleep(2)             # 2 秒待つ
    yellow.value(0)           #  黄を消す

    red.value(1)              #  赤を点ける
    time.sleep(5)             # 5 秒待つ
    red.value(0)              #  赤を消す → また緑に戻る
```

色付き枠 : **while**

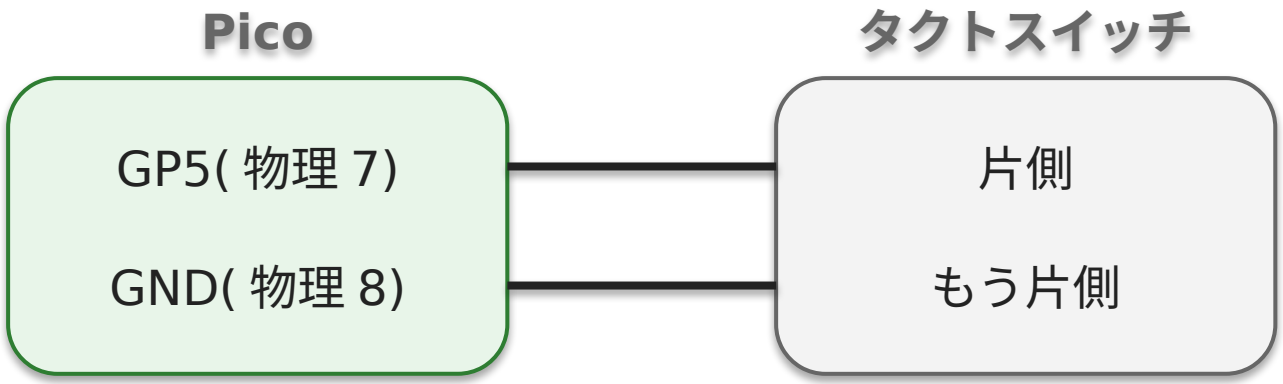
信号機コードの読み解き

コード	意味
`Pin(2, Pin.OUT)`	GP2 を **出力** に設定
`def all_off():` + `for`	全部消す **関数**
`for name, p in ...`	1 個ずつ点灯 → **配線と色が合っているか確認**
`while True:`	緑 → 黄 → 赤 を **くり返す**

★ 実習 : 秒数を変えてみよう !(例 : 緑 3 秒・黄 1 秒・赤 3 秒)

タクトスイッチのしくみ

- **押している間だけ** 電気が通るスイッチ
- 足は 4 本 → **対角線の足** を使うと確実



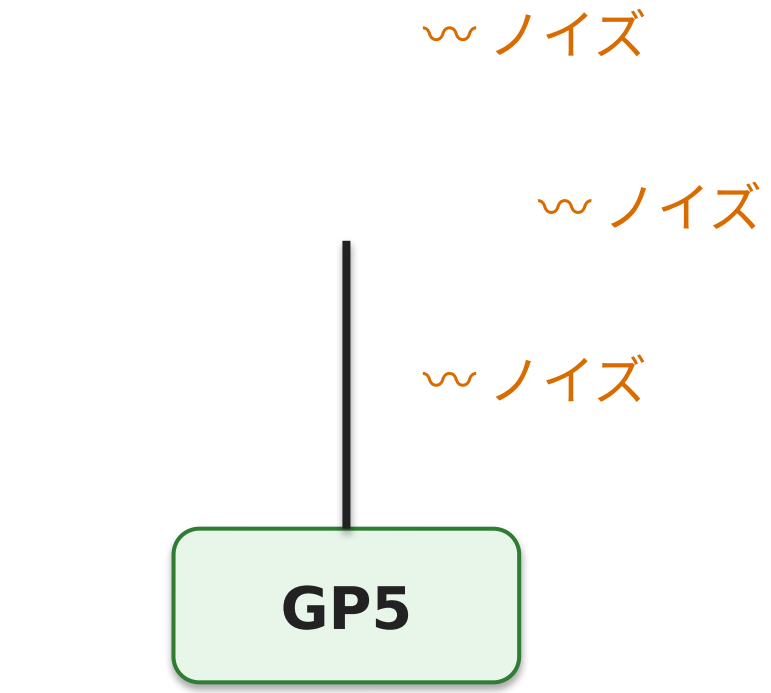
★ 配布したピン配置図で、 GP 番号と物理ピン番号の両方を確かめよう

何もつながっていない入力は「ふらふら」する

・押していないとき、 GP5 は **どこにもつながっていない** (**フローティング**)

・ HIGH か LOW か決まらない

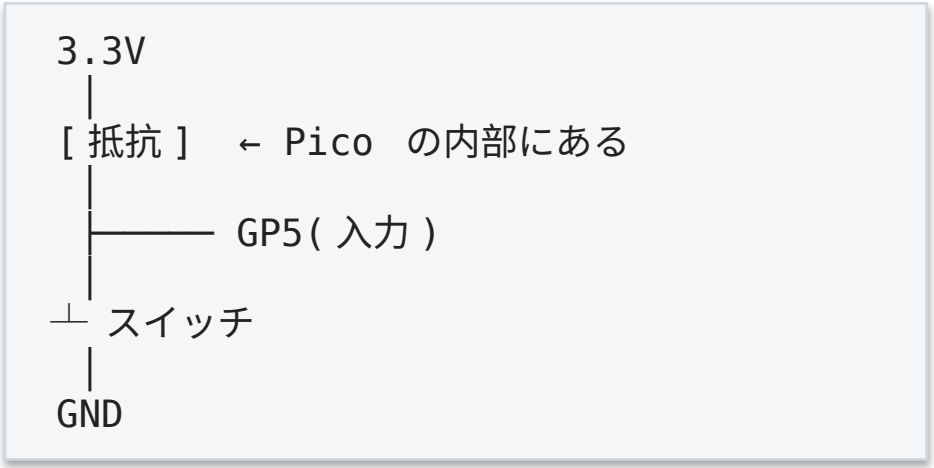
・触っただけで値が変わることも



線がアンテナのようにノイズを拾う

⚠ 値が「 1? 0? 1? 0? 」とふらふら…

プルアップ抵抗とは



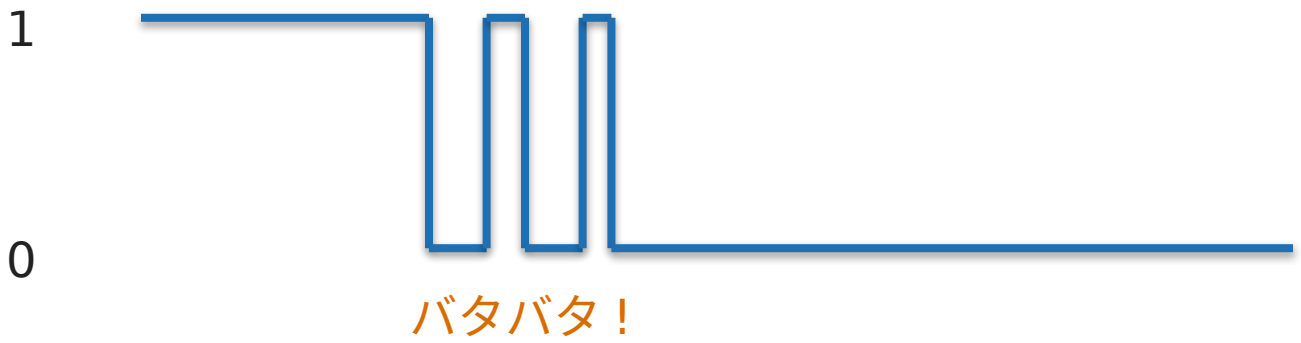
ボタン	つながる先	値
押していない	3.3V	**HIGH(1)**
押している	GND	**LOW(0)**

- `Pin.PULL_UP` で内部のプルアップ抵抗を使える (外付け不要)
- 抵抗があるので、押しても 3.3V と GND が**ショートしない**

⚠ **「押すと 0」** になる点が逆なので注意！

チャタリング

- 押した瞬間に接点が **細かく ON/OFF をくり返す**
- １回押したのに、何回も押したように見える



対策：**少し待って (20ms)**
もう一度確認する

押しボタン信号 (ポーリング版) コード (1/3)

```
# ---- 必要な道具を読み込む ----
from machine import Pin
import time

# ---- ピンの設定 ----
red    = Pin(2, Pin.OUT)   # GP2: 赤 ( 出力 )
yellow = Pin(3, Pin.OUT)   # GP3: 黄 ( 出力 )
green  = Pin(4, Pin.OUT)   # GP4: 緑 ( 出力 )

# GP5 をボタン用の「入力 (IN)」にする
# PULL_UP: 内部のプルアップ抵抗を使う
#   押していない → 1(HIGH)
#   押している → 0(LOW) ← 押すと 0 になる点に注意！
button = Pin(5, Pin.IN, Pin.PULL_UP)

# ---- 関数：全部の LED を消す ----
def all_off():
    for p in (red, yellow, green):
        p.value(0)
```

→ 次のスライドに続く

色付き枠 : for def

押しボタン信号 (ポーリング版) コード (2/3)

```
# ----  スタート時の状態 ----
all_off()
green.value(1)              #  車用の信号は緑で待つ
print(" 待機中 ( 緑 )")

# ----  メインの繰り返し ----
while True:
    #  ボタンの状態を読む。 0  なら押されている
    if button.value() == 0:
        time.sleep_ms(20)                  # 20 ミリ秒待つ ( チャタリング対策 )
        if button.value() == 0:            # 20 ミリ秒後もまだ押されていたら「本当に押した」と判断
            print(" ボタン押下 ")
            time.sleep(3)                  #  押してからすぐには変わらず、 3 秒待つ

            #  緑 → 黄
            green.value(0)
            yellow.value(1)
            print(" 黄 ")
            time.sleep(2)                  #  黄は 2 秒
```

→ 次のスライドに続く

色付き枠 : while if

押しボタン信号 (ポーリング版) コード (3/3)

```
            #  黄 → 赤
            yellow.value(0)
            red.value(1)
            print(" 赤 ( 歩行者 OK)")
            time.sleep(5)                  #  赤は 5 秒 ( この間に歩行者がわたる )

            #  赤 → 緑 ( 元に戻る )
            red.value(0)
            green.value(1)
            print(" 緑に復帰 ")

            #  ボタンが押しっぱなしなら、離されるまでここで待つ
            # ( 待たないと、すぐに次のサイクルが始まってしまう )
            while button.value() == 0:
                time.sleep_ms(10)

    # 10 ミリ秒ごとにボタンを見に行く ( これがポーリング )
    time.sleep_ms(10)
```

色付き枠 : **while**

ポーリング版の読み解き

- **ポーリング** ＝「何度も見に行って確認する」方式
- `if` の中に `if(` **入れ子** `)`→ チャタリング対策
- `while button.value() == 0:` → **離すまで待つ**

⚠ 実験：黄・赤の最中にボタンを押してみよう
→ まったく反応しない！`sleep` 中は **見ていない** から

割り込み (irq) とは

ポーリング
郵便受けを **何度も見に行く**

割り込み
インターホンが鳴ったら 出る

- 割り込み ＝ 「何かが起きたら **知らせてもらう**」方式
- `Pin.IRQ_FALLING`:HIGH → LOW に変わった瞬間 (＝ **押した瞬間**)
- 割り込みで呼ばれる関数 (**ハンドラ**) は短く：**フラグを立てるだけ**

押しボタン信号 (irq 版) コード (1/4)

```
# ----  必要な道具を読み込む ----
from machine import Pin
import time

# ----  ピンの設定 ( ポーリング版と同じ ) ----
red    = Pin(2, Pin.OUT)
yellow = Pin(3, Pin.OUT)
green  = Pin(4, Pin.OUT)
button = Pin(5, Pin.IN, Pin.PULL_UP)   #  押すと 0  になる

# ----  フラグ ( 目印 ) の変数 ----
#  ボタンが押されたら True  にする。最初は「押されていない」 = False
pressed = False

# ----  割り込みで呼ばれる関数 ( ハンドラ ) ----
#  ボタンが押された瞬間に、 Pico  が自動でこの関数を呼んでくれる
def on_press(pin):
    global pressed     #  関数の外にある pressed  を書き換えるための宣言
    pressed = True     #  「押されたよ」と目印を立てるだけ ( 短く書くのが大事 )
```

→ 次のスライドに続く

色付き枠 : def

押しボタン信号 (irq 版) コード (2/4)

```
# ---- 割り込みの登録 ----
# IRQ_FALLING:1 → 0 に変わった瞬間 (= 押した瞬間 ) に on_press を呼ぶ
button.irq(trigger=Pin.IRQ_FALLING, handler=on_press)

# ---- 関数 : 全部の LED を消す ----
def all_off():
    for p in (red, yellow, green):
        p.value(0)

# ---- スタート時の状態 ----
all_off()
green.value(1)
print("待機中 ( 緑 )")

# ---- メインの繰り返し ----
while True:
    # ボタンを直接見に行かず、目印 (pressed) を確認する
    if pressed:
        pressed = False             # 目印を下ろしておく
        print("ボタン押下")
        time.sleep_ms(3000)         # 3 秒待つ
```

→ 次のスライドに続く

色付き枠 : while if for def

押しボタン信号 (irq 版) コード (3/4)

```
        #  緑 → 黄
        green.value(0)
        yellow.value(1)
        print(" 黄 ")
        time.sleep_ms(2000)          #  黄は 2 秒

        #  黄 → 赤
        yellow.value(0)
        red.value(1)
        print(" 赤 ( 歩行者 OK)")
        time.sleep_ms(5000)          #  赤は 5 秒
```

→ 次のスライドに続く

押しボタン信号 (irq 版) コード (4/4)

```
        #  赤 → 緑
        red.value(0)
        green.value(1)
        print(" 緑に復帰 ")

        #  黄・赤の間に押されていても、割り込みで目印が立っている
        #  ここで False  に戻すと「その押下は無視」
        #  この行を消すと「予約」として次のサイクルが始まる
        pressed = False

    time.sleep_ms(50)                # 0.05 秒ごとに目印を確認
```

irq 版の読み解きとポーリング版との比較

- `global pressed`: 関数の外の変数を書き換えるための宣言

	ポーリング	割り込み (irq)
しくみ	何度も見に行く	起きたら知らせてもらう
sleep 中の押下	気づかない	**フラグとして残る**
わかりやすさ	わかりやすい	少し難しい

★ 発展：最後の `pressed = False` を消すと？
→「**予約**」として次のサイクルにつながる

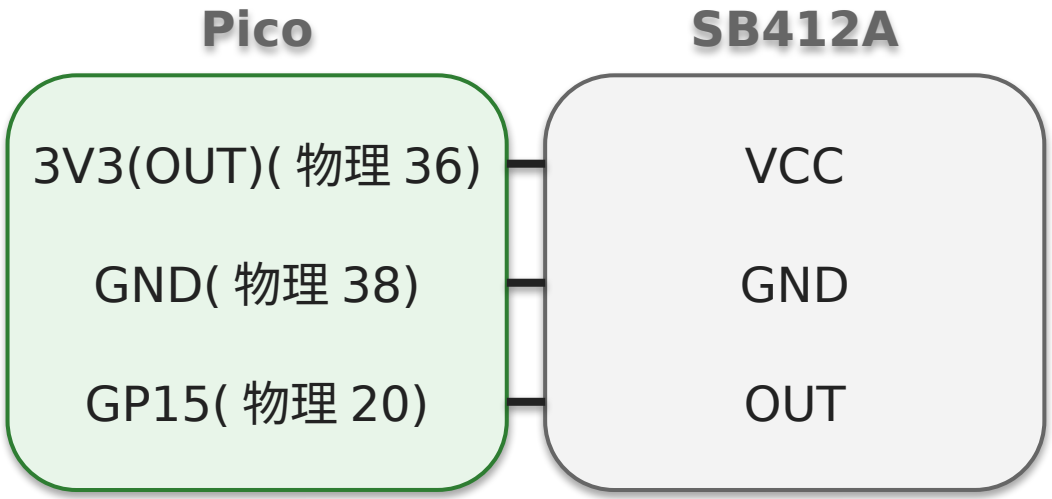
人感センサーのしくみ

・人の体から出る **赤外線 (熱) の変化** を感じ取る

・検知すると OUT が **HIGH** 、いなくなると **LOW**

・電源を入れて数十秒は不安定 (**ウォームアップ**)

・検知後しばらく HIGH が続く (**ホールド時間**)



⚠ 基板の印字で **VCC / OUT / GND の並び** を必ず確認！ 配布したピン配置図で、 GP 番号と物理ピン番号の両方を確かめよう

人感センサーのコード (1/2)

```
# ---- 必要な道具を読み込む ----
from machine import Pin
import time

# ---- ピンの設定 ----
# GP15 を人感センサーの入力にする
# PULL_DOWN: 何もつながっていないときは 0(LOW) に固定する
pir = Pin(15, Pin.IN, Pin.PULL_DOWN)

# Pico 本体に付いている LED を出力にする ( 番号ではなく "LED" と名前で指定する )
led = Pin("LED", Pin.OUT)

# ---- ウォームアップ ----
# 電源を入れてすぐはセンサーが不安定なので 30 秒待つ
print(" ウォームアップ中 ...")
time.sleep(30)
print(" 開始 ")

# ---- 前回の値を覚えておく変数 ----
prev = 0   # 最初は「検知なし」として始める
```

→ 次のスライドに続く

人感センサーのコード (2/2)

```
# ----  メインの繰り返し ----
while True:
    now = pir.value()           #  今のセンサーの値 (1: 検知、 0: 検知なし )

    #  前回と値が変わったときだけ表示する
    # ( 毎回表示すると Shell  が文字だらけになるため )
    if now != prev:
        if now == 1:
            print(" 検知 !")
        else:
            print(" 検知なし ")
        led.value(now)          #  検知中は本体 LED を点ける
        prev = now              #  今の値を「前回の値」として覚える

    time.sleep_ms(100)          # 0.1 秒ごとに確認
```

色付き枠 : while if

人感センサーコードの読み解き

	プルアップ	プルダウン
何もないとき	3.3V に引っ張る → **1**	GND に引っ張る → **0**
書き方	`Pin.PULL_UP`	`Pin.PULL_DOWN`

- `prev`(前回の値) と `now`(今の値) をくらべて、**変化したときだけ** 表示する

★ 実習 : 検知したら信号機を赤にしてみよう！

超音波センサーのしくみ

- 人に聞こえない高い音 (**40kHz**) を出し、**返ってくるまでの時間** を測る
- コウモリやイルカと同じ考え方
- **Trig**: 音を出す合図 /**Echo**: 返ってくるまでの時間だけ HIGH



音の速さから距離を求める

音速 約 **340m/ 秒** (気温 15℃ 付近。正確には **331.5 + 0.6 × 気温**)μs = 100 万分の 1 秒

① 単位をそろえる

式	m/ 秒 → cm/ 秒 → cm/μs
数値を入れる	340m/ 秒 = 34000cm/ 秒
答え	**0.034cm/μs**

② 往復なので 2 で割る

式	距離 = 時間 × 0.034 ÷ 2
数値を入れる	= 時間 × 0.017 = 時間 ÷ 58.8…
答え	**距離 ≒ 時間 ÷ 58(t / 58)**

③ 例 :Echo が 580μs

式	距離 = 時間 ÷ 58
数値を入れる	580 ÷ 58
答え	**10cm**

5V と 3.3V の問題

HC-SR04
Echo から **5V**

5V そのままは ×

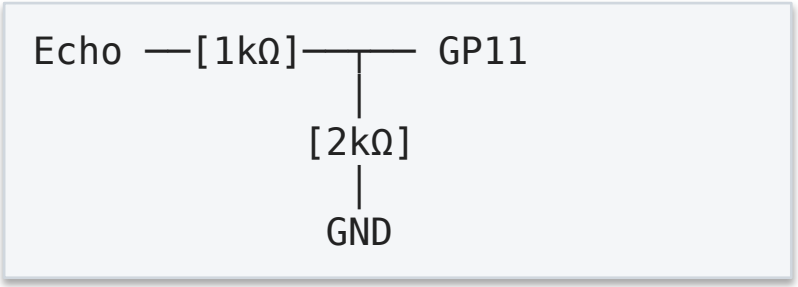
Pico
入力は **3.3V まで**

⚠ 5V をそのまま入れると **壊れるおそれ**
→ 電圧を下げる **「分圧回路」** を使う

抵抗で電圧を分ける (分圧)

出力電圧 ＝ 入力電圧 × R2 ÷(R1 + R2)

式	5V × R2 ÷(R1 + R2)
数値を入れる	5V × 2kΩ ÷(1kΩ + 2kΩ)= 5 × 2/3
答え	**≒ 3.3V**

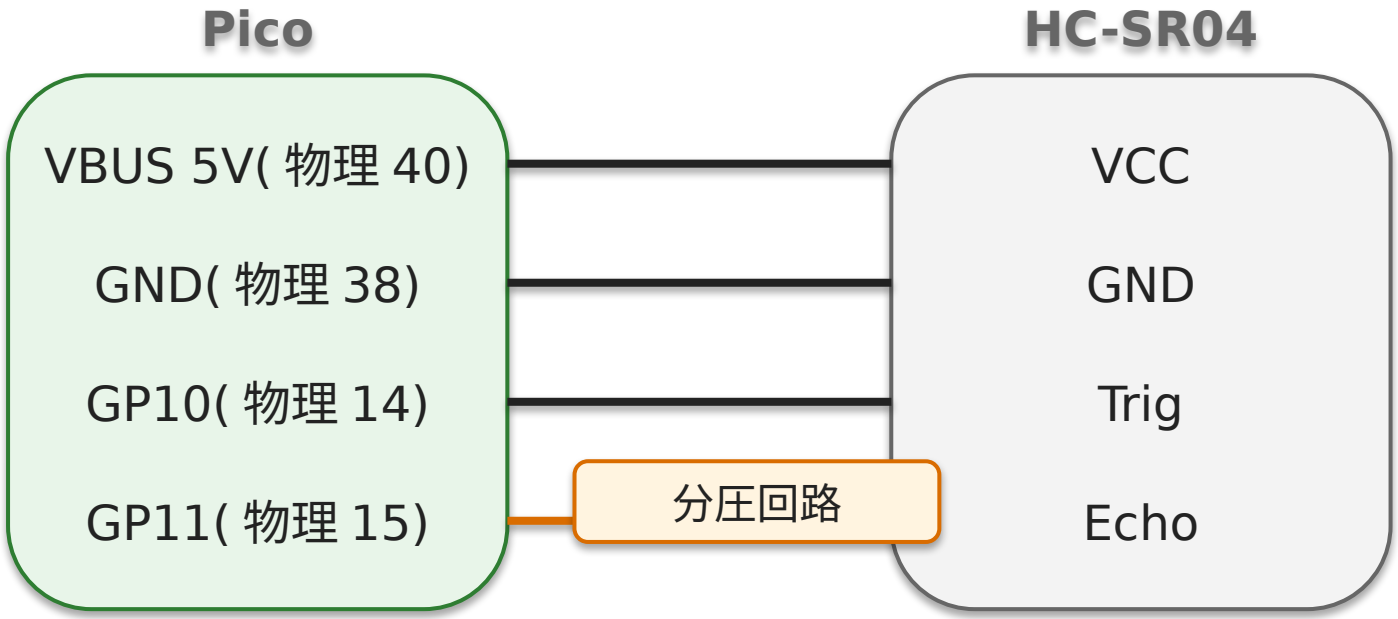


R1 = 1kΩ(Echo 側)

R2 = 2kΩ(GND 側)

- たとえ : 水の流れを 2 つの坂に分けると、途中の高さは坂の長さで決まる
- **Trig** は 3.3V の信号で動くので **直結 OK**
- 3.3V 対応品 (**HC-SR04P** など) なら分圧は不要

超音波センサーの配線まとめ



- VCC は **VBUS(5V)**
- Echo は **分圧回路経由**
- Trig は直結 OK

★ 配布したピン配置図で、 GP 番号と物理ピン番号の両方を確かめよう

超音波センサーのコード (1/3)

```
# ----  必要な道具を読み込む ----
# time_pulse_us: ピンが HIGH  になっている時間を µs(100 万分の 1 秒 ) 単位で測る道具
from machine import Pin, time_pulse_us
import time

# ----  ピンの設定 ----
trig = Pin(10, Pin.OUT)    # GP10:Trig( 音を出す合図を送る → 出力 )
echo = Pin(11, Pin.IN)     # GP11:Echo( 返ってきた時間を受け取る → 入力 )
                           # ※  Echo  は分圧回路 (1kΩ ・ 2kΩ) を通してつなぐ

# ----  関数 : 距離を測って cm  で返す ----
def read_cm():
    #  合図の前に、いったん 0  にして落ち着かせる
    trig.value(0)
    time.sleep_us(2)

    # 10µs  だけ 1  にする → センサーが超音波を出す
    trig.value(1)
    time.sleep_us(10)
    trig.value(0)
```

→ 次のスライドに続く

色付き枠 : **def**

超音波センサーのコード (2/3)

```
    # Echo が 1 になっている時間 ( 音が往復した時間 ) を測る
    # 30000µs(=30 ミリ秒 ) 待っても返ってこなければあきらめる
    t = time_pulse_us(echo, 1, 30000)

    # 測れなかったとき、t はマイナスの値になる
    if t < 0:
        return None         # None =「値なし」を返す

    # 時間 (µs) を距離 (cm) に変える
    # 音速 0.034cm/µs × 時間 ÷ 2( 往復なので )≒ 時間 ÷ 58
    return t / 58
```

→ 次のスライドに続く

色付き枠 : if

超音波センサーのコード (3/3)

```
# ---- メインの繰り返し ----
while True:
    d = read_cm()           #  距離を測る

    if d is None:
        print(" 測定不可 ")
    else:
        # {:.1f} は「小数第 1 位まで表示する」という意味
        print("{:.1f} cm".format(d))

    time.sleep_ms(200)      # 0.2 秒ごとに測る
```

色付き枠 : while if

超音波センサーコードの読み解き

コード	意味
Trig を 10µs だけ HIGH	「音を出して！」の合図
`time_pulse_us(echo, 1, 30000)`	Echo が HIGH の時間を µs で測る。30ms 以上はあきらめる
`if t < 0: return None`	測れなかったときは「値なし」を返す
`return t / 58`	時間 → 距離 (cm) の計算

⚠ 測定範囲は約 **2 ～ 400cm** 。近すぎる・斜めの面・やわらかい物は苦手

今日のまとめ

制御構造	順次・`while`・`if`・`for`・関数
電気の基本	LED には電流制限抵抗 (220Ω 程度)/ プルアップ・プルダウン / 分圧
センサー	人感 (赤外線)/ 超音波 (音速 × 時間 ÷ 2)
ボタン	ポーリングと割り込み

チャレンジ課題

1. 超音波で **10cm 以内** に近づいたら信号機を **赤** にする
2. 人感センサーが反応したら **黄を点滅** させる
3. ボタンを押したら超音波で距離を測り、**距離に応じて信号の色を変える**